CHAPTER 0.5 — PREREQUISITE

Time and Space Complexity

Understand this before writing a single algorithm.

This chapter has one job: make Big-O notation completely clear. No maths degree needed. No scary formulas. Just plain English, real examples in Python, and by the end you will know exactly what O(1), O(n), O(n^2), O(log n) and O(n log n) mean and how to spot them in any code you write.

Section 1

What is time complexity and why should you care?

Imagine you write code that works perfectly on your laptop with 10 items. You feel good. You submit it in an interview. The interviewer says: 'What if the input has 10 million items?' You freeze.

That question is about time complexity. It is asking: how does your code's speed change as the input grows? This is the single most important question in every coding interview.

Time complexity is not about seconds. It is not about your laptop speed or Python being slow. It is about the relationship between input size and the number of operations your code performs.

> **The core idea in one sentence**
>
> Time complexity answers: if I double the input size, does my code take double the time? Four times? The same time? The answer to that question is what Big-O notation captures.

Space complexity is the same idea but for memory instead of time. How much extra memory does your code use as input grows? We will cover space complexity in Section 6 after time complexity is solid.

Section 2

A real example before any notation

Forget computers for a moment. You have a physical notebook with names written on 1000 pages, one name per page. Your friend gives you a name and asks: is this name in the notebook?

You have two options:

1. Flip through every page from the beginning until you find the name or run out of pages.
2. The notebook is sorted alphabetically. You open the middle. Name comes before middle? Check the first half. After? Check the second half. Keep halving until found.

Option 1: worst case you check all 1000 pages. If there are 2000 names tomorrow, you check 2000. Input doubles, work doubles.

Option 2: 1000 pages takes about 10 checks. 2000 pages takes about 11 checks. Input doubles but work barely increases.

This is the difference between O(n) and O(log n). One grows with input. One barely grows at all. We are now going to make this precise.

Section 3

O(1) — Constant time

O(1) means: no matter how big the input is, your code always does the same number of operations. The input size does not matter at all.

Think of it like a light switch. Whether your house has 10 rooms or 1000 rooms, flipping the switch on the wall next to you takes exactly one action. The size of the house is irrelevant.

Python — O(1) examples

```
my_list = [10, 20, 30, 40, 50]

# Accessing by index — always one operation
first = my_list[0]     # O(1)
last  = my_list[-1]    # O(1)
third = my_list[2]     # O(1)
# Does not matter if list has 5 or 5 million elements

my_dict = {'name': 'Alice', 'age': 25}

# Dictionary lookup — always one operation
name = my_dict['name']         # O(1)
exists = 'age' in my_dict      # O(1)

# Checking length
size = len(my_list)            # O(1)
# Python stores the length, does not count every time
```

How to recognise O(1) in code

No loops. No recursion. Just direct access or a fixed number of steps. If you can count the operations and the count does not change with input size, it is O(1).

Section 4

O(n) — Linear time

O(n) means: as input size grows, your work grows at the same rate. Double the input, double the work. Triple the input, triple the work. It is a straight line relationship.

n is just a variable that represents the size of your input. If your list has 100 elements, n = 100. If it has 1 million elements, n = 1,000,000.

Python — O(n) examples

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

# Example 1: find the maximum
def find_max(nums):
    max_val = nums[0]
    for num in nums:      # visits every element once
        if num > max_val:
            max_val = num
    return max_val
# If nums has 10 elements  -> 10 comparisons
# If nums has 1000 elements -> 1000 comparisons
# Work grows with n. This is O(n).

# Example 2: sum all elements
def total(nums):
    result = 0
    for num in nums:      # one pass through all elements
        result += num
    return result
# Also O(n)

# Example 3: check if value exists
def contains(nums, target):
    for num in nums:      # worst case: check every element
        if num == target:
            return True
    return False
# O(n) in worst case
```

> **How to recognise O(n) in code**
>
> One loop that goes through all elements once. The loop runs n times where n is your input size. Each iteration does a fixed amount of work (no nested loops inside).

Notice the pattern: one loop over all elements = O(n). This is the most common complexity you will see and write.

Section 5

O(n²) — Quadratic time

O(n²) means: as input grows, your work grows much faster. Double the input, the work becomes four times as much. Triple the input, nine times the work.

This happens when you have a loop inside a loop, both going over the input. For each of the n elements, you do n more operations. n times n equals n².

Python — O(n²) example

```
numbers = [3, 1, 4, 1, 5, 9]

# Find all pairs that add up to 10
def find_pairs(nums, target):
    pairs = []
    for i in range(len(nums)):        # runs n times
        for j in range(i+1, len(nums)):   # runs ~n times
            if nums[i] + nums[j] == target:
                pairs.append((nums[i], nums[j]))
    return pairs

# For n=6:  outer runs 6 times, inner runs ~5,4,3,2,1 times
# Total operations: roughly 6*6 = 36 = n²

# For n=100:   about 10,000 operations
# For n=1000:  about 1,000,000 operations
# For n=10000: about 100,000,000 operations  <- very slow
```

Why O(n²) is dangerous in interviews

For n=10,000 elements, an O(n²) solution does 100 million operations. Most online judges timeout at around 100 million operations per second. When an interviewer says 'can you do better?' they almost always mean 'can you replace this O(n²) with O(n) or O(n log n)?'

How to recognise O(n²) in code

A loop inside a loop, where both loops iterate over the same input. The signal: you see 'for i in range(len(nums))' and then 'for j in range(len(nums))' directly inside it.

Section 6

O(log n) — Logarithmic time

This one scares most beginners because of the word 'logarithm'. Forget the maths definition. Here is the practical meaning:

O(log n) means: every step of your code cuts the remaining problem in half. You never look at all n elements. You keep halving until done.

Remember the notebook example from Section 2? That was O(log n). With 1000 pages you needed about 10 checks. With 1 million pages you need only about 20 checks. The input grew by 1000x but the work only doubled. That is logarithmic.

Python — O(log n) example: binary search

```
def binary_search(sorted_list, target):
    left  = 0
    right = len(sorted_list) - 1

    while left <= right:
        mid = (left + right) // 2

        if sorted_list[mid] == target:
            return mid          # found it
        elif sorted_list[mid] < target:
            left = mid + 1      # target is in right half
        else:
            right = mid - 1     # target is in left half

    return -1   # not found

# Each iteration cuts the search space in half
# n=1000  -> at most 10 iterations
# n=1M    -> at most 20 iterations
# n=1B    -> at most 30 iterations
```

Let us make the halving concrete with a small example:

Step	Search space remaining	What happens
Start	1000 elements	Check middle element (position 500)
Step 1	500 elements	Target not at 500, eliminate half
Step 2	250 elements	Check middle of remaining half
Step 3	125 elements	Keep halving
Step 4	63 elements	Keep halving
Step 5	32 elements	Almost there
Step 10	1 element	Found or not found. Done in 10 steps.

The log n intuition you must memorise

log base 2 of n answers the question: how many times can I divide n by 2 before reaching 1?

n = 8 -> divide by 2 three times (8->4->2->1) -> log(8) = 3

n = 1000 -> about 10 times -> log(1000) is about 10

n = 1,000,000 -> about 20 times -> log(1,000,000) is about 20

This is why O(log n) algorithms are so fast. Even for massive inputs, the number of operations stays tiny.

How to recognise O(log n) in code

The key signal: your loop variable does not go up by 1 each time. It doubles, or your search space halves each iteration. Binary search is the classic example. You will learn it in Chapter 5.

Section 7

O(n log n) — The sorting complexity

O(n log n) sits between O(n) and O(n²). It is slower than linear but much faster than quadratic. The most important thing to know about it: this is the complexity of good sorting algorithms like merge sort and Python's built-in sort.

The intuition: you do a log n depth operation, but you do it for each of the n elements. n times log n = n log n.

Python — O(n log n)

```
numbers = [5, 2, 8, 1, 9, 3]

# Python's built-in sort is O(n log n)
numbers.sort()         # sorts in place
# [1, 2, 3, 5, 8, 9]

# sorted() also O(n log n), returns new list
sorted_nums = sorted(numbers)

# In interviews: if you sort first then do one pass,
# total complexity is O(n log n) + O(n) = O(n log n)
# The larger term dominates
```

You do not need to implement O(n log n) sorting right now. Just know that when you call .sort() in Python, that is O(n log n), not O(n) and not O(n²).

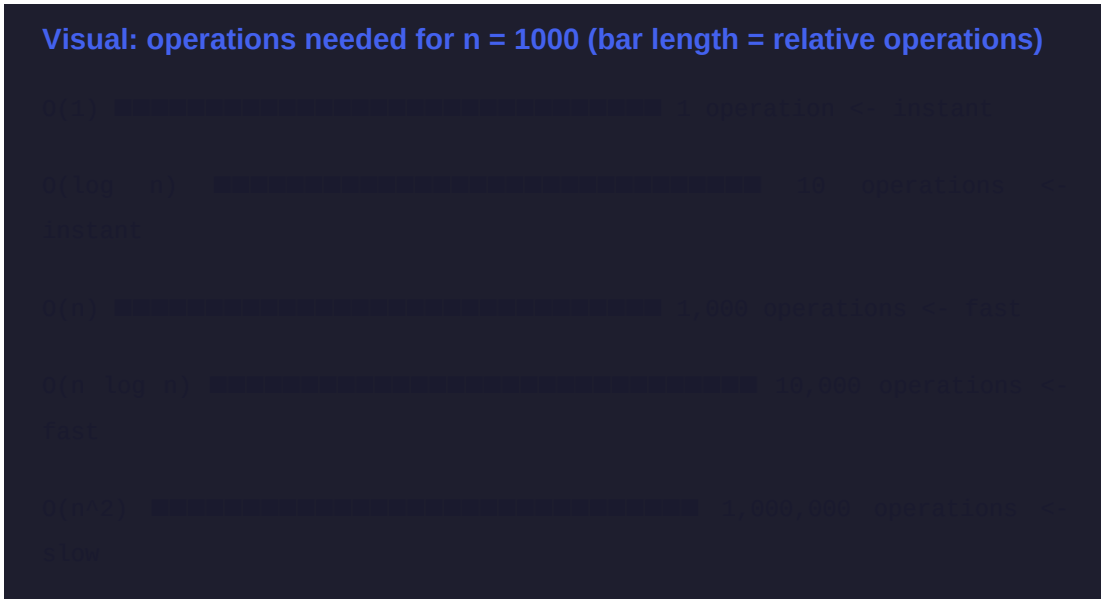
Section 8

Comparing all complexities side by side

Here is how many operations each complexity requires for different input sizes. Read this table carefully. It shows you exactly why complexity matters.

Complexity	n = 10	n = 100	n = 1,000	n = 1,000,000
O(1)	1	1	1	1
O(log n)	3	7	10	20
O(n)	10	100	1,000	1,000,000
O(n log n)	30	700	10,000	20,000,000
O(n^2)	100	10,000	1,000,000	1,000,000,000,000

Look at n = 1,000,000. O(1) does 1 operation. O(log n) does 20. O(n) does one million. O(n^2) does one trillion. A computer doing one billion operations per second would take 1 millisecond for O(n) but 11 days for O(n^2). Same input. Same computer. Different algorithm.



Section 9

How to calculate complexity of any code you write

Follow these four rules every time you want to find the complexity of code.

Rule 1: Count the loops

One loop over n elements = O(n). Loop inside a loop over n elements = O(n^2). This covers 80 percent of what you will encounter.

Rule 1 examples

```
# One loop -> O(n)
for i in range(n):        # runs n times
    do_something()        # O(1) work each time

# Loop inside loop -> O(n²)
for i in range(n):        # runs n times
    for j in range(n):    # runs n times for each i
        do_something()    # n * n = n²
```

Rule 2: Drop the constants

O(2n) is written as O(n). O(3n²) is written as O(n²). We drop the constant multipliers because they do not change the growth pattern. Two loops that each run n times = O(2n) = O(n).

Rule 2 examples

```
# Two separate loops, both O(n)
for i in range(n):      # n operations
    do_something()

for j in range(n):      # n more operations
    do_something_else()

# Total: 2n operations = O(2n) = O(n)
# We drop the 2. It is still O(n).
```

Rule 3: Keep only the dominant term

If your code is O(n² + n), you write O(n²). For large n, the n² term is so much bigger than n that n becomes irrelevant. Always keep the term that grows fastest.

Rule 3 examples

```
# First part: O(n²)
for i in range(n):
    for j in range(n):
        do_something()

# Second part: O(n)
for k in range(n):
    do_something_else()

# Total: O(n²) + O(n) = O(n² + n) = O(n²)
# The n² dominates. Drop the n.
```

Rule 4: Halving = log n

If your code keeps dividing the problem in half at each step, it is O(log n). You will see this in binary search and certain tree operations.

Rule 4 example

```
# Each iteration cuts the range in half
left, right = 0, n
while left < right:
    mid = (left + right) // 2
    if condition:
        right = mid      # half eliminated
    else:
        left = mid + 1  # other half eliminated

# Halving each time -> O(log n)
```

Quick complexity calculator

See a single loop over n elements? -> O(n)

See a loop inside a loop over n elements? -> $O(n^2)$

See the problem halving each step? -> O(log n)

See .sort() or sorted()? -> O(n log n)

See direct index access or dict lookup? -> O(1)

Section 10

Space complexity

Space complexity measures how much extra memory your code uses as input grows. The rules are the same as time complexity but instead of counting operations you count memory allocated.

Python — space complexity examples

```python
# O(1) space — no extra memory that grows with input
def find_max(nums):
    max_val = nums[0]     # one variable, always
    for num in nums:
        if num > max_val:
            max_val = num
    return max_val
# Extra memory used: just max_val. Does not grow. O(1) space.

# O(n) space — extra memory grows with input
def double_all(nums):
    result = []           # new list, grows with input
    for num in nums:
        result.append(num * 2)
    return result
# Extra memory: result list with n elements. O(n) space.

# O(n) space — dictionary grows with input
def count_freq(nums):
    freq = {}             # can have up to n keys
    for num in nums:
        freq[num] = freq.get(num, 0) + 1
    return freq
# Extra memory: freq dict. O(n) space.
```

The key distinction

The input itself does not count toward space complexity. We only count extra memory your code creates. If your function takes a list of n elements and creates nothing new, that is O(1) space even though the input takes O(n) memory.

In most interview problems, the interviewer cares more about time complexity than space complexity. But always be ready to state both. The standard format is: 'Time complexity O(n), space complexity O(1)'.

Section 11

Practice: what is the complexity of this code?

Look at each code block below. Before reading the answer, figure out the time complexity yourself. This is the skill interviews test.

Problem 1

What is the time and space complexity?

```
def mystery_one(nums):
    total = 0
    for num in nums:
        total += num
    return total
```

Answer to Problem 1

Time: O(n). One loop visits every element once.

Space: O(1). Only one extra variable 'total'. Does not grow with input.

Problem 2

What is the time and space complexity?

```
def mystery_two(nums):
    result = []
    for i in range(len(nums)):
        for j in range(len(nums)):
            if nums[i] + nums[j] == 0:
                result.append((nums[i], nums[j]))
    return result
```

Answer to Problem 2

Time: $O(n^2)$. Loop inside a loop, both going over n elements.

Space: $O(n^2)$ in worst case. result list could hold up to n^2 pairs.

Problem 3

What is the time and space complexity?

```
def mystery_three(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
```

Answer to Problem 3

Time: O(n). One loop. 'in seen' is O(1) because seen is a set.

Space: O(n). The set 'seen' can grow up to n elements.

This is the efficient way to find duplicates. You will write this pattern many times in this book.

Problem 4

What is the time and space complexity?

```
def mystery_four(nums):
    nums.sort()              # sort first
    left  = 0
    right = len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == 0:
            return True
        elif s < 0:
            left += 1
        else:
            right -= 1
    return False
```

Answer to Problem 4

Time: O(n log n). The sort dominates. The while loop after is O(n) but O(n log n) + O(n) = O(n log n). Sorting always dominates a single pass.

Space: O(1). No extra data structures. Just two pointer variables.

This is the two-pointer pattern you will master in Chapter 3.

Section 12

Your Big-O cheat sheet

Keep this page open while you study. Refer to it every time you write a solution until complexity analysis becomes automatic.

Complexity	Name	You see this when...	Common example
O(1)	Constant	Direct access, dict lookup, set membership	list[i], dict[key], x in set
O(log n)	Logarithmic	Problem halves each step	Binary search
O(n)	Linear	One loop over all elements	Find max, sum, linear search
O(n log n)	Linearithmic	Sort then scan	sorted(), .sort(), merge sort
O(n^2)	Quadratic	Loop inside a loop over same input	Brute force pair problems

Python operation	Time complexity	Why
my_list[i]	O(1)	Direct index access
my_list.append(x)	O(1)*	*Amortized. Occasional resize is O(n).
my_list.insert(0,x)	O(n)	Shifts all elements right
x in my_list	O(n)	Checks every element
x in my_set	O(1)	Hash lookup
my_dict[key]	O(1)	Hash lookup
key in my_dict	O(1)	Hash lookup
my_list.sort()	O(n log n)	Timsort algorithm
sorted(my_list)	O(n log n)	Same algorithm, new list
len(my_list)	O(1)	Python stores length internally
my_list[a:b]	O(b-a)	Creates a copy of the slice

Before moving to Chapter 1

Answer these without looking anything up. If you cannot answer one, go back and re-read that section.

1. What does O(1) mean in plain English?
2. What does O(n) mean in plain English?
3. If a list has 1 million elements, roughly how many steps does O(log n) take?
4. You see a loop inside a loop, both over n elements. What is the complexity?
5. Why is 'x in my_list' O(n) but 'x in my_set' O(1)?
6. What is the complexity of Python's .sort() method?
7. Your code does $O(n^2)$ work then O(n) work. What do you report as the total?
8. What is space complexity? Give one example of O(1) space and one of O(n) space.
9. You see 'left, right = 0, n' and a while loop that halves the range each time. What complexity?

10. Why does $O(n^2)$ become dangerous for large inputs but $O(n \log n)$ stays manageable?

If you can answer all 10

You are ready for Chapter 1. Time complexity will feel natural as you write each solution because you now understand what you are measuring.

If you are stuck on any of them

That is fine. Go back to that specific section. Run the code. Change the input sizes and print the results. Understanding comes from doing, not just reading.

End of Chapter 0.5 — Next: Chapter 1: Arrays, Strings and Big-O in practice